# Grupo 2 · Módulo 1: Búsqueda y alta de reservas

Su parte: situaciones marcadas B y las trampas (T). Para cada una: elijan el patrón, escriban por qué y un patrón que descartaron con el motivo. Si ningún patrón hace falta, escriban «ninguno» y fundamenten.

| N° | Situación | Patrón elegido | ¿Por qué? / Descartado y por qué |
|---|---|---|---|
| P2 | La app funciona en Argentina, Brasil y Europa. Cada mercado tiene su moneda, formato de fecha, impuestos y documento (DNI o pasaporte) que deben combinar entre sí, y nunca se mezclan los de dos mercados. | Abstract Factory | Por qué: Permite crear familias de objetos relacionados (moneda, fecha, impuestos) que deben usarse juntos dependiendo de la región, garantizando que no se mezclen.<br><br>Descartado: Builder. Se descarta porque el problema no es la complejidad de construir un objeto paso a paso, sino asegurar que las piezas pertenezcan a la misma familia (mercado).<br><br>Descartaría Factory Method para el caso P2 principalmente porque no garantiza la consistencia entre familias de objetos. |
| P4 | Cada temporada se crea el paquete «Bariloche 7 noches» copiando el del año anterior, con sus vuelos, hoteles y excursiones, y después se ajustan fechas y precios. | Builder | Por qué: Permite construir un objeto complejo (el paquete con sus listas de vuelos, hoteles y excursiones) paso a paso. Se puede usar un Director o un Builder concreto que tenga la "receta" prearmada del paquete Bariloche, permitiendo al cliente ajustar los atributos variables (fechas y precios) antes de pedir el objeto final con el método build().<br><br>Descartado: Factory Method. Se descarta porque instanciar un paquete con tantas partes internas mediante un solo método de creación |

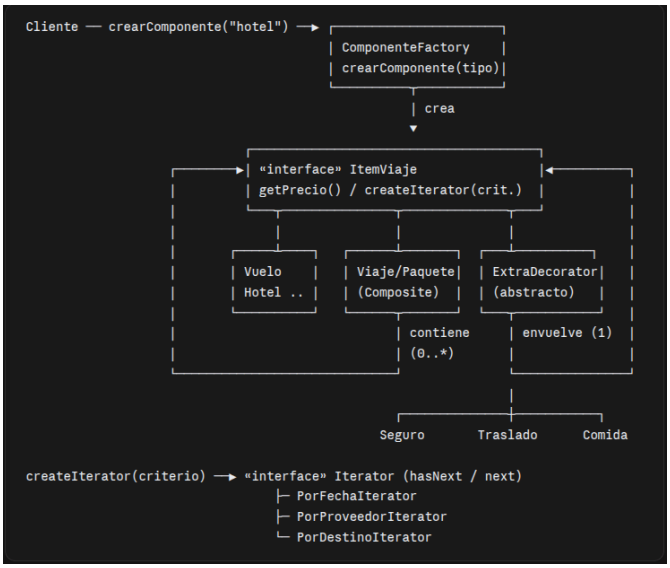
| | | | |
|---|---|---|---|
| | | | requeriría pasarle demasiados parámetros a la vez en el constructor, volviendo el código rígido y difícil de leer. Builder maneja mejor la complejidad de ensamblar todas esas partes. |
| T1 (T) | El nombre «ViajaYa» aparece en el pie de pantalla y cambia una vez cada diez años. | Ninguno | Por qué: Es una simple constante global o un valor en un archivo de configuración (.properties, .env). |

# Grupo 2 · Escenario B: Armado de paquetes

Combinen 3 o 4 patrones para resolver todo lo siguiente:

5. Armar el árbol paquete, viaje y componente con el precio total.
6. Recorrer por fecha, por proveedor o por destino sin saber cómo se guardan.
7. Los componentes tienen extras combinables: seguro, traslado, comida.
8. Crear componentes a partir de un tipo («vuelo», «hotel») sin que el cliente conozca las clases.

Entregable: un dibujo de cajas y flechas con los patrones elegidos y una frase por cada uno. El cambio de último momento llega más tarde.

| Pregunta | Respuesta del grupo |
|---|---|
| Patrones elegidos (3 o 4) | **Composite**, **Iterator**, **Decorator**, **Factory Method** |
| Dibujo (cajas y flechas) |  <br>• **Composite (5):** paquete, viaje y componente comparten la interfaz `ItemViaje` con `getPrecio()`, y los contenedores suman recursivamente a sus hijos, así que el precio total sale sin distinguir hojas de contenedores.<br>• **Iterator (6):** cada recorrido (por fecha, proveedor o destino) es un iterador distinto, y el cliente solo usa `hasNext()` y `next()` sin saber cómo se guarda el árbol.<br>• **Decorator (7):** cada extra (`Seguro`, `Traslado`, `Comida`) envuelve a un `ItemViaje` y le suma su costo, y se pueden apilar en cualquier combinación: `new Comida(new Seguro(hotel))`.<br>• **Factory Method (8):** el cliente pide `crearComponente("vuelo")` y recibe un `ItemViaje` sin conocer las clases concretas. |
| Dos patrones que se combinan y cómo | **Composite + Decorator:** ambos implementan `ItemViaje`, así que un componente decorado (ej. hotel + seguro + comida) se puede colgar del paquete como cualquier otro nodo. El `getPrecio()` del paquete suma el precio ya decorado de cada hijo, y ni el paquete ni el cliente saben que hay extras. Además, **Factory Method + Decorator:** la fábrica entrega el componente base y el cliente lo envuelve con los extras que quiera. |

| | |
|---|---|
| Un patrón que descartaron y por qué | **Builder.** Sirve para armar un objeto con muchas partes en un solo paso, pero acá los extras se suman a componentes que ya existen y en cualquier combinación. Con Builder habría que decidir todo al construir, mientras que Decorator permite agregar o quitar un extra sobre un objeto ya creado sin tocar su clase. |
| Cuando llegue el giro: ¿qué cambia y<br>qué NO se toca? | ● **Componente nuevo (ej. «crucero», «alquiler de auto»):**<br>○ **Cambia:** se agrega la clase nueva que implementa `ItemViaje` y una línea en `ComponenteFactory`.<br>○ **No se toca:** Composite, iteradores ni decoradores.<br>● **Criterio de recorrido nuevo (ej. por precio):**<br>○ **Cambia:** se agrega un `PorPrecioIterator`.<br>○ **No se toca:** el árbol, la fábrica ni los demás iteradores.<br>● **Extra nuevo (ej. «excursión opcional», «upgrade de habitación»):**<br>○ **Cambia:** se agrega una clase que extiende `ExtraDecorator`.<br>○ **No se toca:** componentes, paquete, iteradores ni los extras existentes. |

# Grupo 2 · Tarjeta de revisión

Revisen el diseño de otro grupo. Grupo revisado: ___ Escenario: _____

| Pregunta | Sí / No / Dudoso | Comentario |
|---|---|---|
| 1. ¿Cada patrón resuelve un problema real del escenario? | | |
| 2. ¿Hay algún patrón de más (sobreingeniería)? | | |
| 3. ¿Se puede aplicar el giro sin tocar el código que ya funcionaba? | | |
| 4. ¿Qué preguntaría un compañero que no estuvo en el grupo? | | |

Ticket de salida (papelito aparte): un patrón que hoy elegiste y no usarías en tu trabajo real, y por qué.